

AWS Lightsail Server Setup and Deployment Report for Inventory Management System

Prepared for

Inventory Management System Deployment

Prepared by

Victor Munene

Date

July 5, 2026

1. Introduction

This report outlines the complete step-by-step procedure for setting up a new AWS Lightsail server environment and deploying the Inventory Management System. The objective is to ensure that all necessary software, dependencies, services, and configurations are installed correctly and functioning properly.

The deployment environment includes:

- Debian 12 operating system
- Apache Web Server
- PHP 8.3
- MariaDB/MySQL Database
- phpMyAdmin
- Composer
- Git
- GitHub repository integration
- PDF generation support (Dompdf)
- Proper file permissions and security configurations

This guide aims to create a fully functional and stable server environment while minimizing future configuration issues.

2. Creating an AWS Lightsail Instance

The first step involves creating a new virtual server instance within AWS Lightsail.

Recommended configuration:

Platform:

- Linux/Unix

Blueprint:

- OS Only

Operating System:

- Debian 12 (Bookworm)

Recommended instance plan:

- 2 GB RAM
- 2 vCPU
- 60 GB SSD

Instance name:

inventory-system

After instance creation:

1. Create a Static IP
2. Attach the Static IP to the instance
3. Record the assigned IP address

Purpose:

A Static IP ensures that the server address remains unchanged even after restarting the server.

3. Connecting to the Server

Connect to the server through SSH:

```
ssh admin@YOUR_SERVER_IP
```

Purpose:

Allows remote access to manage and configure the server.

4. Updating System Packages

Run:

```
sudo apt update && sudo apt upgrade -y
```

Explanation:

- `sudo`: Executes commands with administrator privileges
- `apt update`: Updates package information from repositories
- `apt upgrade`: Installs latest package versions
- `-y`: Automatically confirms installation

Purpose:

Ensures all system packages are up to date with the latest security updates and bug fixes.

5. Installing Apache Web Server

Install Apache:

```
sudo apt install apache2 -y
```

Enable Apache service:

```
sudo systemctl enable apache2
```

Start Apache:

```
sudo systemctl start apache2
```

Verify service status:

```
sudo systemctl status apache2
```

Expected output:

active (running)

Purpose:

Apache serves web pages and processes incoming user requests.

Testing:

Open browser:

http://YOUR_SERVER_IP

Expected output:

Apache2 Debian Default Page

6. Configuring Firewall Rules

Within AWS Lightsail networking settings, configure:

Protocol Port Purpose

SSH	22	Remote access
-----	----	---------------

HTTP	80	Website access
------	----	----------------

HTTPS	443	Secure website access
-------	-----	-----------------------

Purpose:

Allows users and administrators to communicate with the server.

7. Installing PHP 8.3

Install prerequisites:

```
sudo apt install -y lib-release ca-certificates apt-transport-https software-properties-common curl gnupg2
```

Purpose:

Installs required tools for adding external repositories.

Add PHP repository key:

```
curl -sSL https://packages.sury.org/php/apt.gpg | sudo gpg --dearmor -o /usr/share/keyrings/php.gpg
```

Add repository:

```
echo "deb [signed-by=/usr/share/keyrings/php.gpg] https://packages.sury.org/php/ $(lsb_release -sc) main" | sudo tee /etc/apt/sources.list.d/php.list
```

Update package lists:

```
sudo apt update
```

Install PHP:

```
sudo apt install -y php8.3 libapache2-mod-php8.3 php8.3-cli php8.3-common php8.3-mysql php8.3-curl php8.3-gd php8.3-mbstring php8.3-xml php8.3-zip php8.3-intl php8.3-bcmath
```

Restart Apache:

```
sudo systemctl restart apache2
```

Verify installation:

```
php -v
```

Expected output:

PHP 8.3.x

Purpose:

PHP processes server-side scripts required by the Inventory System.

8. Installing MariaDB Database Server

Install MariaDB:

```
sudo apt install mariadb-server -y
```

Enable database service:

```
sudo systemctl enable mariadb
```

Start database service:

```
sudo systemctl start mariadb
```

Secure database installation:

```
sudo mysql_secure_installation
```

Recommended selections:

Set root password → Yes

Remove anonymous users → Yes

Disallow remote root login → Yes

Remove test database → Yes

Reload privileges → Yes

Purpose:

Secures the database environment against unauthorized access.

Verify installation:

`sudo mysql`

Exit:

`exit;`

9. Installing phpMyAdmin

Install phpMyAdmin:

`sudo apt install phpmyadmin -y`

Select:

Apache2

Enable PHP extension:

`sudo phpenmod mbstring`

Restart Apache:

`sudo systemctl restart apache2`

Test installation:

`http://YOUR_SERVER_IP/phpmyadmin`

Purpose:

Provides a graphical interface for database management.

10. Installing Composer

Download Composer:

`curl -sS https://getcomposer.org/installer | php`

Move globally:

`sudo mv composer.phar /usr/local/bin/composer`

Verify:

`composer --version`

Purpose:

Composer manages PHP project dependencies.

11. Installing Git

Install Git:

```
sudo apt install git -y
```

Verify:

```
git --version
```

Purpose:

Git enables version control and integration with GitHub repositories.

12. Deploying the Inventory System

Move into web directory:

```
cd /var/www/html
```

Clone repository:

```
sudo git clone YOUR_GITHUB_REPOSITORY_URL
```

Move into project:

```
cd inventory_system
```

Install project dependencies:

```
composer install
```

Install PDF generation package:

```
composer require dompdf/dompdf
```

Purpose:

Downloads all required project libraries and packages.

13. Configuring Permissions

Set ownership:

```
sudo chown -R www-data:www-data /var/www/html/inventory_system
```

Set permissions:

```
sudo chmod -R 755 /var/www/html/inventory_system
```

Purpose:

Allows Apache to read and execute project files safely.

14. Creating Database and Importing Data

Access database:

```
sudo mysql -u root -p
```

Create database:

```
CREATE DATABASE inventory_db;
```

Exit:

```
exit;
```

Import SQL file:

```
mysql -u root -p inventory_db < inventory_db.sql
```

Purpose:

Creates and restores project data.

15. Testing System Functionality

Perform the following checks:

1. Verify home page loads
2. Test login functionality
3. Test registration
4. Verify database connectivity
5. Test file uploads
6. Test PDF generation
7. Test email functionality
8. Test CRUD operations
9. Verify user roles and permissions
10. Review system logs for errors

Purpose:

Ensures that all application components function correctly.

16. Security Measures

Recommended security enhancements:

Enable HTTPS:

```
sudo apt install certbot python3-certbot-apache -y
```

Generate SSL certificate:

```
sudo certbot --apache
```

Enable firewall:

```
sudo ufw allow OpenSSH
```

```
sudo ufw allow Apache Full
```

```
sudo ufw enable
```

Enable automatic updates:

```
sudo apt install unattended-upgrades -y
```

Purpose:

Protects the server from security vulnerabilities and unauthorized access.

17. Conclusion

Following this procedure ensures that the AWS Lightsail server is fully configured for hosting the Inventory Management System. The process installs all necessary software dependencies, configures the web and database services, establishes security measures, and validates that the system functions correctly. Proper implementation of these steps minimizes deployment errors and improves reliability, security, and maintainability.